



PolyPulse

Surveillance des marchés « boostés » de Polymarket — deux micro-services et un dashboard temps réel pour suivre cotes, volumes et récompenses de liquidité.

BINÔME

Dayssam Bakaar

Kais Hammache

CLASSE

LSI 1 · APP

Ingé 2

STACK

TypeScript · Node 22

Express · Jest · Docker · PostgreSQL

ÉCHÉANCE

21 juin

Remise sur Moodle

DÉPÔT GIT

github.com/2soum/PolyPulse

01 Présentation & cas d'usage

PolyPulse surveille les marchés de prédiction *boostés* de Polymarket : ceux qui distribuent des récompenses aux apporteurs de liquidité.

L'utilisateur peut :

- **découvrir** les marchés boostés les plus actifs (tri par volume 24h) ;
- les **ajouter à une watchlist** persistée en base de données ;
- suivre un **dashboard** : cote « Oui », variation 24h, volume, **boost score**, courbe d'évolution (snapshots) et répartition par catégorie.

Pourquoi ce sujet pour un projet DevOps ? Le flux impose **deux services back** qui communiquent par **HTTP** (*watch* → *poly*), et *poly* appelle l'**API externe Polymarket Gamma**. Ces **deux frontières réseau** rendent les **mocks web** exigés par le sujet réellement nécessaires — à deux niveaux.

```
# Détection « boosté » et score d'attractivité
boosted = rewardsMinSize > 0 OU holdingRewardsEnabled
score   = 40·(rewardsMinSize>0) + 40·(holdingRewards) + 20·min(1, rewardsMaxSpread/5) # [0,100]
```

02 Architecture logicielle

2.1 — Vue micro-services



`watch-service` porte le domaine (watchlist, snapshots, KPIs) ; `poly-service` est un service de calcul sans état encapsulant Polymarket.

2.2 — Architecture en couches

watch-service	poly-service
Controller · <code>watchController.ts</code>	Controller · <code>marketController.ts</code>
Services · <code>watchService.ts</code> , <code>polyGateway.ts</code>	Services · <code>marketService.ts</code> , <code>polymarketClient.ts</code> , <code>marketNormalizer.ts</code>
Data · <code>WatchRepository</code> , <code>SnapshotRepository</code> (Postgres)	Data · <code>CategoryRepository</code>

Règle de dépendance. Chaque couche dépend uniquement de la couche inférieure, via une **interface** (`WatchRepository` , `PolymarketClient` , `PolyGateway` ...). Les implémentations en mémoire et PostgreSQL sont interchangeables sans impact sur les couches supérieures.

2.3 — Séquence : ajouter un marché puis rafraîchir

```
Dashboard → watch-service : POST /api/watchlist { marketId }
watch-service → poly-service : GET /api/markets/{id}
poly-service → Polymarket Gamma : GET /markets/{id}
watch-service : enregistre watchlist + 1er snapshot [PostgreSQL]
Dashboard → watch-service : GET /api/dashboard
watch-service : snapshot par marché + agrégation KPIs
watch-service → Dashboard : { kpis, rows[] (sparklines) }
```

03 Tests, couverture & qualité

78

TESTS VERTS

97.9%

LIGNES · POLY

97.1%

LIGNES · WATCH

4

COUCHES TESTÉES

Couverture de code

SERVICE	TESTS	STATEMENTS	BRANCHES	FUNCTIONS	LIGNES
poly-service	41	98.16 %	91.89 %	100 %	97.93 %
watch-service	37	97.27 %	94.33 %	100 %	97.12 %

Couverture lignes globale (230 / 236 lignes exécutables)

Des seuils minimaux sont imposés dans `jest.config.js` (`coverageThreshold` : 80 % branches / 85 % lignes). La CI échoue sous ces seuils — la qualité est verrouillée automatiquement.

Stratégie de test par couche

COUCHE	TYPE	OUTILS	EXEMPLE
Data	Unitaire	Jest	<code>categoryRepository.test.ts</code>
Services	Unitaire + mocks	Jest,nock	<code>watchService.test.ts</code>
Controller	Intégration HTTP	Supertest,nock	<code>watchController.test.ts</code>
Inter-service	Mock web	nock	<code>polyGateway.test.ts</code>

Mocks web

- `poly` → Polymarket Gamma (API externe)
- `watch` → `poly` (appel inter-service)

→ tests isolés et déterministes, sans réseau.

Qualité logicielle

- TypeScript `strict` (+ `noUnused*`)
- ESLint 0 erreur
- Prettier + `.editorconfig`
- Erreurs typées → codes HTTP
- Découplage par interfaces

04 Conteneurisation & intégration continue

Docker (≥ 2 services back)

`docker-compose.yml` orchestre 4 conteneurs : `watch-service` , `poly-service` , `postgres` et `frontend` . Chaque service back a un `Dockerfile` multi-stage (build TypeScript → image `node:22-alpine`) avec `HEALTHCHECK` .

```
make up # docker compose up --build → dashboard http://localhost:8080
```

Pipeline CI — GitHub Actions

ÉTAPE	JOB	ACTION
1	test (matrice x2)	<code>npm ci</code> → <code>lint</code> → <code>build</code> → <code>test:coverage</code>
2	test	publication des rapports de couverture en artefacts
3	docker	build des 3 images + <code>docker compose config</code>
4	sonarcloud	couverture des 2 services → <code>SonarQube Cloud Scan</code> (Quality Gate)

Pas de **Continuous Delivery** (hors programme) : le pipeline s'arrête au build / test / qualité.

Conformité au sujet

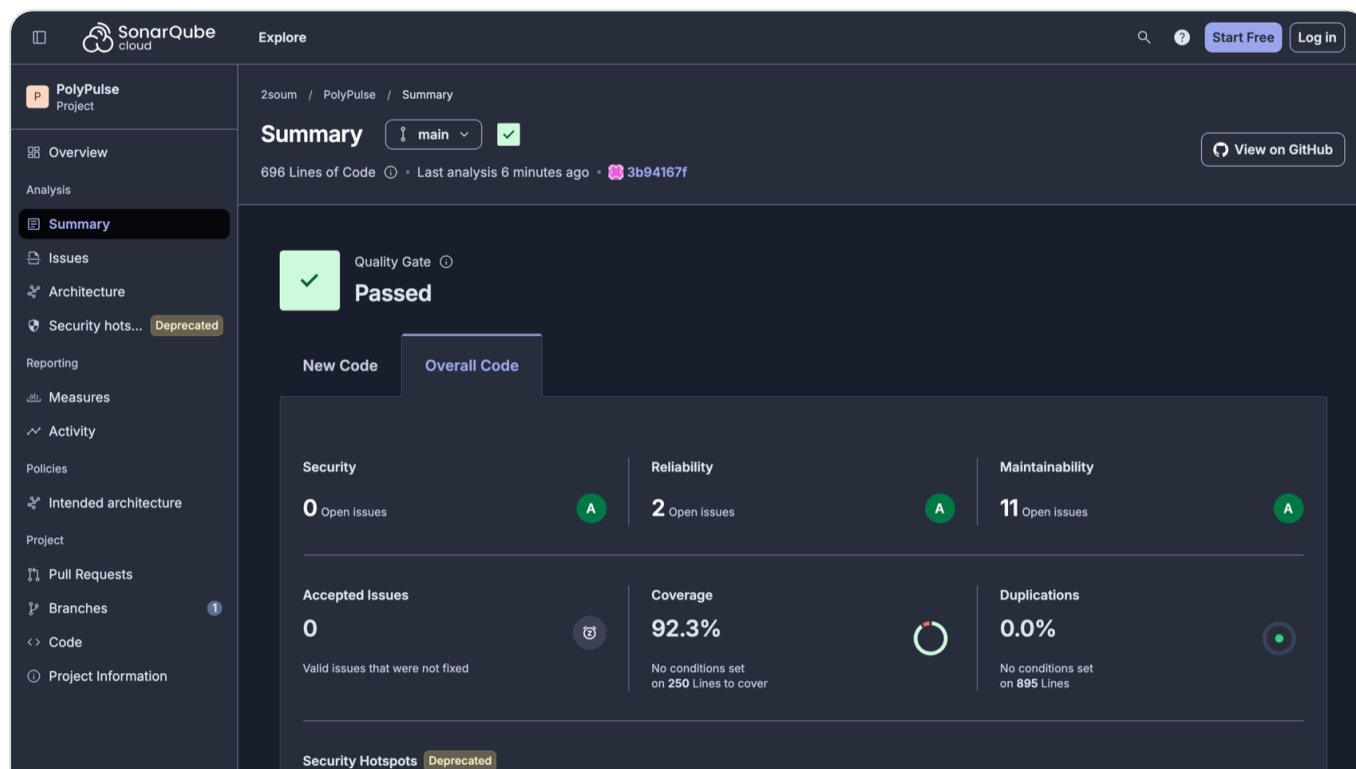
EXIGENCE	STATUT	RÉALISATION
Dépôt Git	✓	dépôt public — github.com/2soum/PolyPulse
Pipeline CI	✓	GitHub Actions (lint·build·test·docker)
Architecture en couches	✓	Data / Services / Controller
≥ 2 services back + Docker	✓	watch-service + poly-service
Tests de toutes les couches	✓	Jest + Supertest +nock
Bonne couverture	✓	~97 % lignes, seuils imposés
Qualité élevée	✓	TS strict · ESLint · Prettier
Analyse qualité (SonarQube Cloud)	✓	Quality Gate Passed · A/A/A · 0 bug · 0 vulnérabilité (§ 5)
Bonus — Front web	✓	dashboard (graphes, filtres)
Bonus — Base de données	✓	PostgreSQL (watchlist + snapshots)

05 Qualité logicielle — SonarQube Cloud

À **chaque push**, la CI lance une analyse **SonarQube Cloud** (avec import de la couverture des deux services) et applique un **Quality Gate** bloquant. C'est le « rapport de qualité » exigé par le sujet — mesuré automatiquement, et public.



Boucle DevOps « analyse → correctif ». Sonar a relevé 2 vulnérabilités `tssecurity:S7044` (un `id` client inséré dans le chemin d'une URL → injection de chemin / SSRF). Corrigées par une **validation en liste blanche** + `encodeURIComponent` (avec tests) → **note de sécurité C → A**.

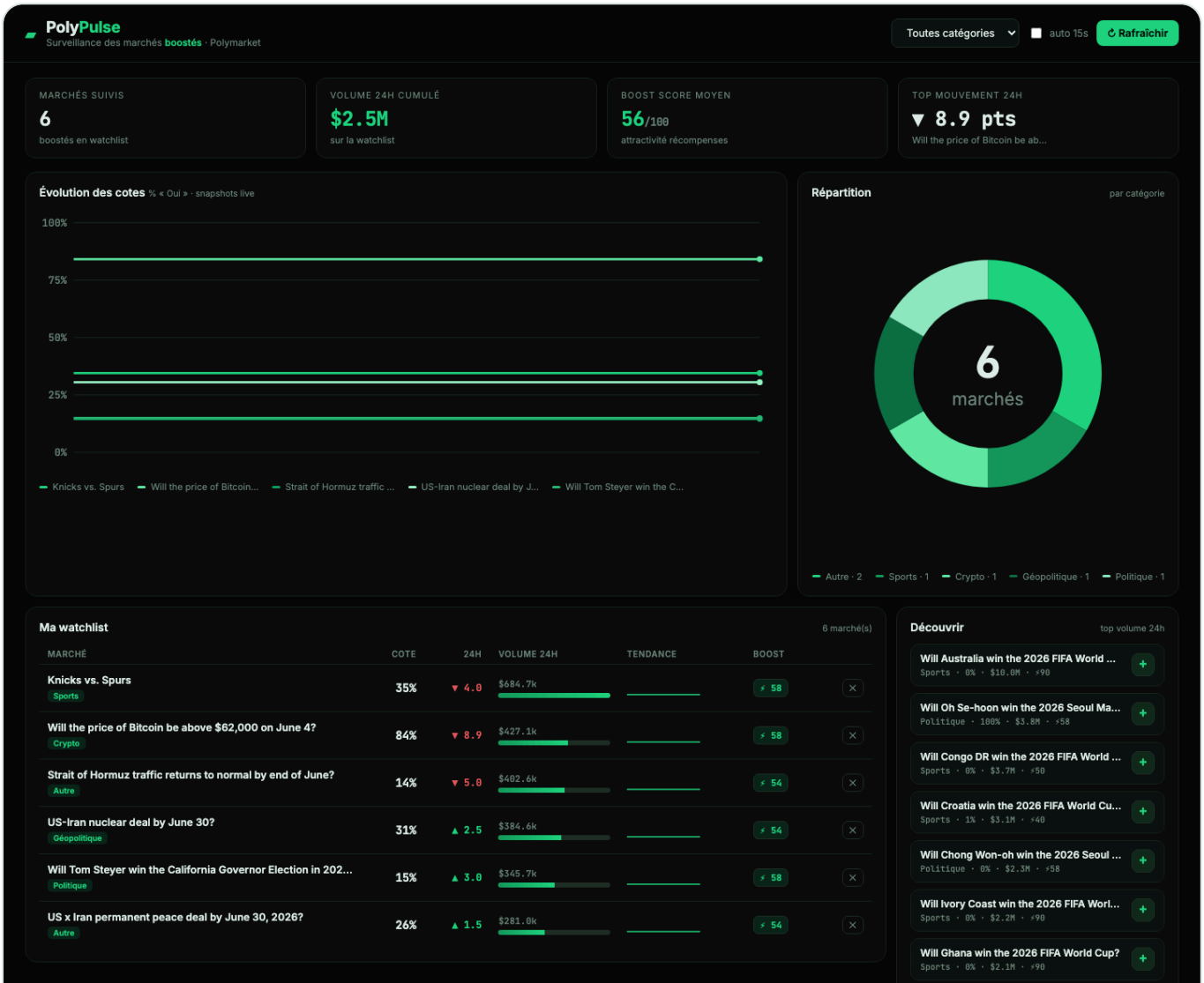


SonarQube Cloud — projet public `2soun_PolyPulse` : Quality Gate **Passed**, notes A/A/A, couverture 92.3 %, duplication 0 %.

06 Le dashboard

Front web (bonus) servi par nginx, thème sobre noir & vert. Il interroge `watch-service` et affiche en temps réel :

- **KPIs** : marchés suivis, volume 24h cumulé, boost score moyen, top mouvement 24h ;
- **Courbe d'évolution des cotes** (multi-séries, à partir des snapshots) ;
- **Donut** de répartition par catégorie ;
- **Table watchlist** : cote, variation colorée, barre de volume, sparkline, boost score ;
- panneau **Découvrir** + filtre par catégorie + auto-refresh.

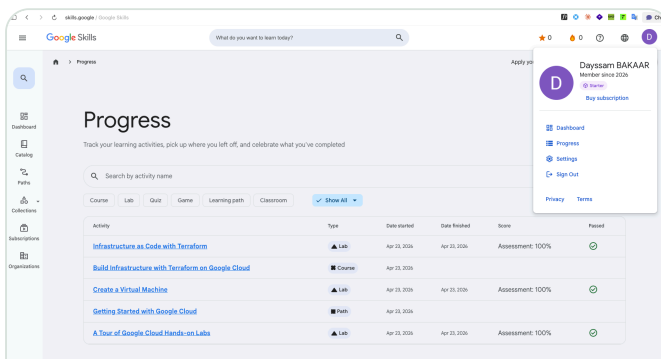


Dashboard PolyPulse — données réelles issues de l'API Polymarket via les deux micro-services.

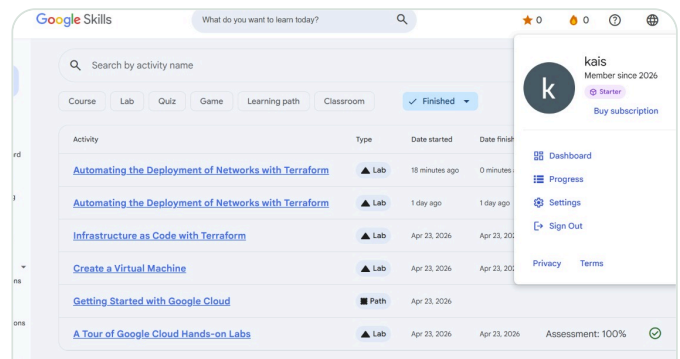
07 Google Labs (Google Skills / Qwiklabs)

Les **deux membres du binôme** ont suivi les parcours et labs Google Cloud (Skills Boost) — chaque évaluation validée à **100 %** :

- *A Tour of Google Cloud Hands-on Labs* — Lab
- *Getting Started with Google Cloud* — Path
- *Create a Virtual Machine* — Lab
- *Infrastructure as Code with Terraform* — Lab
- *Build Infrastructure with Terraform on Google Cloud* — Course
- *Automating the Deployment of Networks with Terraform* — Lab



Dayssam Bakaar — Google Skills « Progress » : labs validés (Assessment 100 %).



Kais Hammache — Google Skills « Progress » : labs validés (Assessment 100 %).

Limites & perspectives

- Dépendance à l'API publique Polymarket Gamma (limites de débit) ; **poly-service** renvoie un **502** en cas d'indisponibilité (testé).
- Les snapshots s'accumulent à l'usage : la courbe d'évolution s'enrichit dans le temps.
- Pistes : alertes sur seuils de boost/volume, historique long terme, authentification.

08 Annexes — rapports bruts

Sorties réelles et reproductibles des outils — les **rapports de tests, de couverture et de qualité** exigés par le sujet. Reproductibles en local (`make coverage` , `make lint`) et rejouées à chaque `push` par la CI.

A — Couverture de code (résumé Jest)

poly-service · 6 suites · 41 tests ✓

Statements	: 98.16% (107/109)
Branches	: 91.89% (68/74)
Functions	: 100% (31/31)
Lines	: 97.93% (95/97)

watch-service · 5 suites · 37 tests ✓

Statements	: 97.27% (143/147)
Branches	: 94.33% (50/53)
Functions	: 100% (40/40)
Lines	: 97.12% (135/139)

B — Couverture par fichier

POLY-SERVICE	STMT	BRANCH	LIGNES
app.ts	100	100	100
config.ts	100	80	100
controller/marketController	95.2	88.9	95
data/categoryRepository	100	100	100
services/marketNormalizer	96.7	90.2	96.2
services/marketService	100	100	100
services/polymarketClient	100	100	100
Tous fichiers	98.2	91.9	97.9

WATCH-SERVICE	STMT	BRANCH	LIGNES
app.ts	100	100	100
config.ts	100	80	100
errors.ts	100	100	100
controller/watchController	89.5	91.7	89.5
data/snapshotRepository	100	100	100
data/watchRepository	100	100	100
services/polyGateway	100	100	100
services/watchService	100	93.8	100
Tous fichiers	97.3	94.3	97.1

Couverture des **fonctions** : 100 % (71/71) sur les deux services · colonnes en %.

C — Tests, qualité & build

CONTRÔLE	COMMANDE	RÉSULTAT
Tests	<code>npm test</code> · Jest + Supertest +nock	78 / 78 ✓ · 11 suites
Lint	<code>npm run lint</code> · ESLint	0 erreur · 0 warning
Compilation	<code>npm run build</code> · tsc strict	OK · 0 erreur
Seuils couverture	<code>coverageThreshold</code> (jest.config)	≥ 80 % br. / 85 % lignes — tenus
Analyse Sonar	<code>sonarqube-scan-action</code> · CI	Quality Gate Passed · A/A/A

Tous ces contrôles sont **bloquants en CI** — la qualité est garantie automatiquement à chaque `push` .